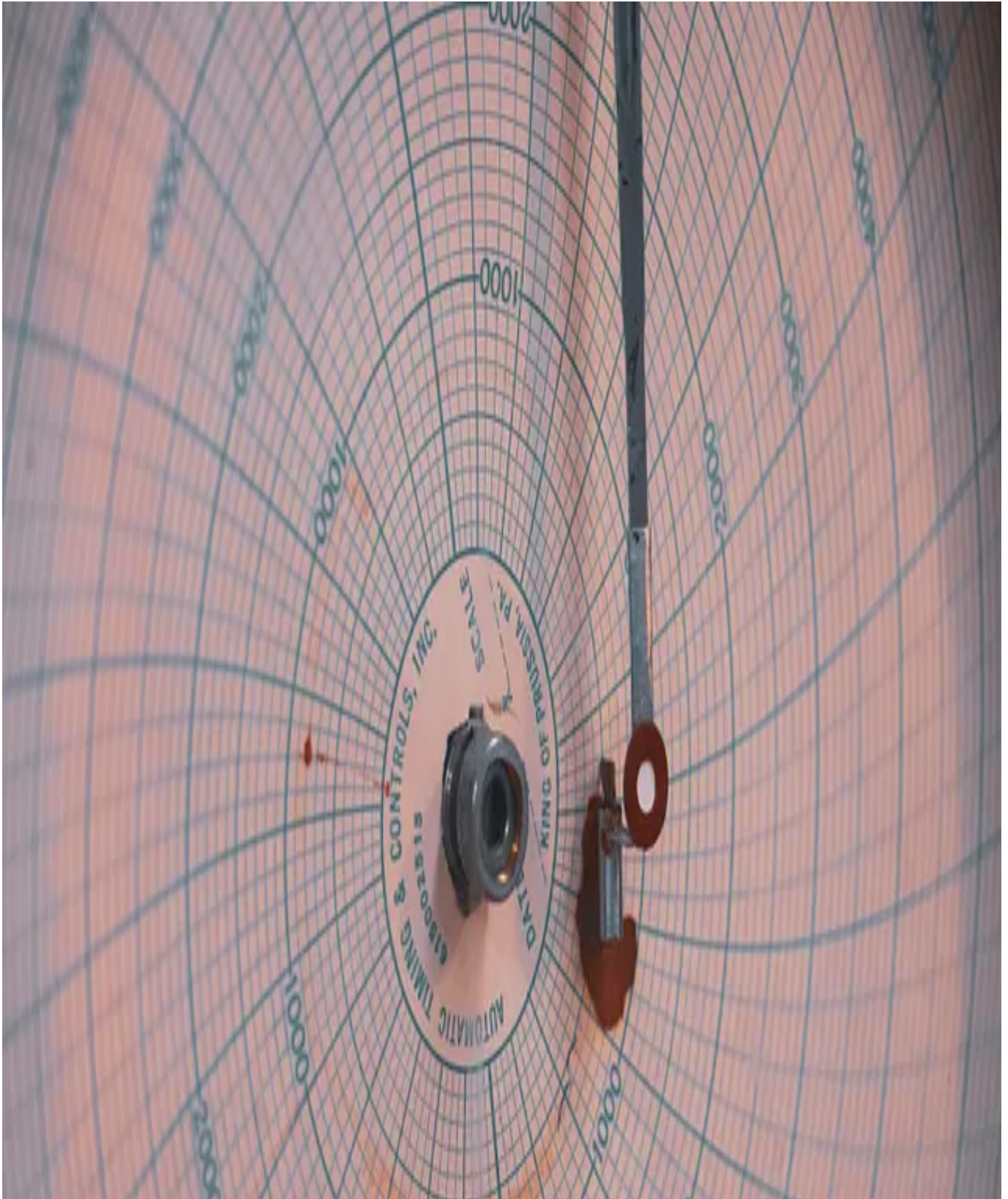


RLCD : Reinforcement Learning for Calibrated Decisions

[Utkarsh Ankit](#)



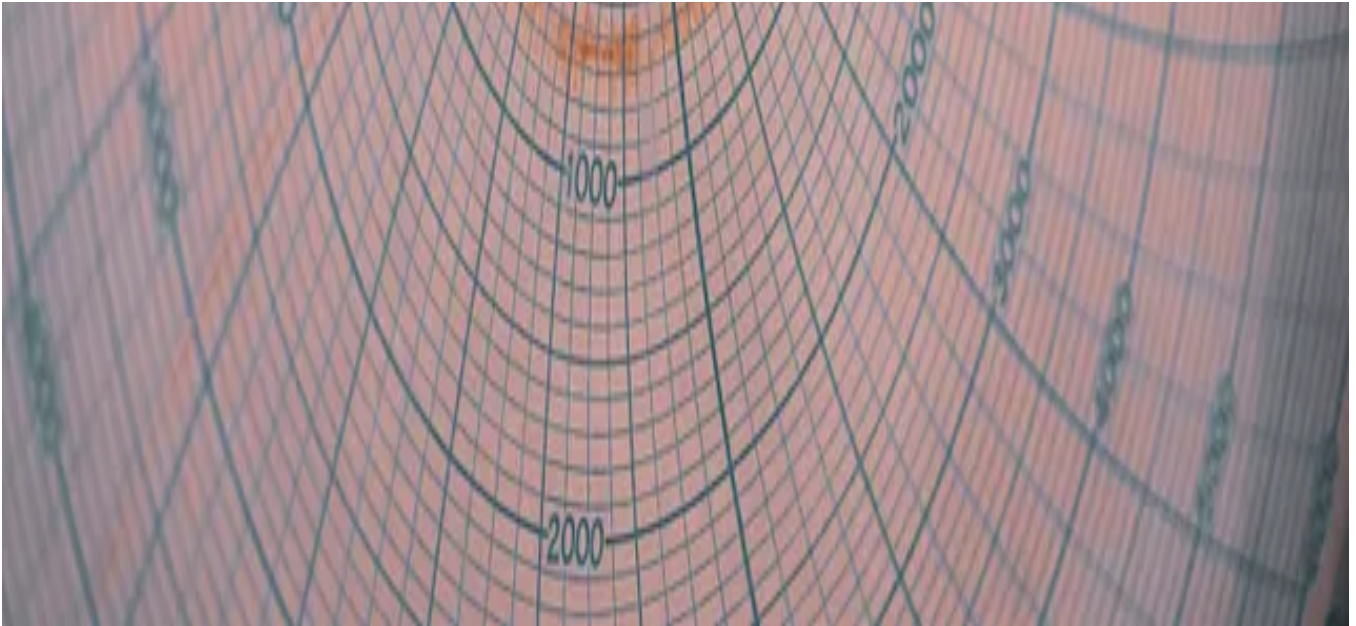


Photo by [Dan Meyers](#) on [Unsplash](#)

A new post-training method devised by TypeSafe AI for their model Jev. Post-training methods are generally an amalgamation of multiple aspects of machine learning systems, and I have observed a lot of creative experiments researchers are doing to make LLMs more robust.

So, let's explore the newest member of this family: RLCD. But before we jump, let's understand Typesafe's first public model, "Jev", and how it differs from a general LLM.

Jev focuses more on automation (software decision-making) rather than generative text generation.

What they did differently:-

- A new architecture
- Parallel Sampler
- A new training method (RLCD)

And they claimed that their architecture is faster and more efficient than state-of-the-art architectures. You can learn more about the Jev architecture [here](#).

Jev's Output

The output of Jev is not text or unstructured data but a JSON object that contains calibrated decision values and probability distributions.

Schema-guaranteed output structure. Source: Chrome for Developers

Jev returns one of the three mathematical objects (`Choice`, `Score`, or `NoUl`) for a decision:-

1. **Choice** Output for Multi-Class Selection

While evaluating a discrete set of categories, it outputs the winning category with a fully calibrated probability vector across all candidates and an overall confidence score.

{

```
"type": "choice",
"selection": "fraud_review",
"confidence": 0.884,
"probabilities": {
  "approve": 0.042,
  "fraud_review": 0.884,
  "decline": 0.074
},
"latency_ms": 84
}
```

2. **Score** Output (Ordinal Scale)

Let's say we have to check the urgency on a 1–5 scale; it returns the probability-weighted expected score with the distribution.

```
{
  "type": "score",
  "value": 4.2,
  "confidence": 0.912,
  "distribution": {
    "1": 0.01,
    "2": 0.03,
    "3": 0.12,
    "4": 0.43,
    "5": 0.41
  },
  "latency_ms": 92
}
```

3. **Noul** Output (Binary)

When we are dealing with binary conditions (T/F or Y/N), it returns a calibrated probability between 0.0 and 1.0 representing likelihood.

```
{  
  "type": "noul",  
  "value": true,  
  "probability": 0.941,  
  "latency_ms": 76  
}
```

So, if we observe the outputs, we see that schema-based errors, missing key hallucinations, or invalid JSON syntax are out of scope because the output is generated by a non-autoregressive output head rather than some sampled text.

Along with that, we are also getting calibrated probabilities that correspond to true ground-truth accuracy.

But here is a catch: we first need to understand what calibration is (skip if you know ECE).

Calibration

Calibration basically means the model's confidence matches how often it's actually right. For example, we built a spam email classification model that says how likely an email is spam. It assigns an 80% spam probability to 100 emails. And when we check:-

- If 80 of 100 are actually spam, we say our model is **perfectly calibrated**.
- If 50 of 100 are actually spam, we say our model is **overconfident**.
- If 95 of 100 are spam, we say our model is **underconfident**.

So, basically, we check if our model is lying or not. We aim that if our model says X%, it should be right X% of the time.

We should not confuse calibration with confidence and accuracy, as they

say different things, but are related:-

- **Accuracy:** How often is the model right? (wrt ground truth)
- **Confidence:** How sure does the model claim to be? (The final Sigmoid/Softmax output)
- **Calibration:** Does the model know how often it is right? (How much the model's confidence matches real-world accuracy)

The problem with standard neural networks or LLMs is that, trained on Cross-Entropy loss, they tend to push softmax probabilities to extreme values (0.999 or 0.001), which makes the model severely overconfident even when it is hallucinating or predicting incorrect values.

*We measure calibration by **Expected Calibration Error (ECE)** and evaluate overall probabilistic quality using **Brier Score**:-*

Expected Calibration Error (ECE):-

- Grouping predictions into bins: eg. Bin 1: 50–60% confidence, Bin 2: 60–70% confidence, ..., Bin 10: 90–100% confidence.
- Calculate accuracy per bin.
- Compute the gap.

$$ECE = \sum (|B_m| / N) * | acc(B_m) - conf(B_m) |$$

Where:

M = Total number of confidence bins

|B_m| = Number of samples in bin m

N = Total number of samples

acc(B_m) = Actual accuracy of predictions in bin m

conf(B_m) = Average confidence assigned to predictions in bin

Then we can plot a **reliability diagram** for better observation.

Reliability Diagram where the yellow-shaded area represents the calibration gap. Source: Kang et al. (2020), licensed under CC BY 4.0.

Brier Score:-

It is used to evaluate calibration (it measures the overall quality of probabilistic predictions: MSE between predicted probabilities and real binary outcomes); lower is better. It leads to maximising the model's score if and only if its reported probability matches its true likelihood of being correct. So, it is a Proper Scoring Rule. Calibration is one component of it, as the Murphy decomposition shows.

Brier Score (Mean Squared Error):

$$BS = (1 / N) * \sum (f_i - o_i)^2$$

where:-

predicted probabilities (f_i)

actual binary outcomes ($o_i \in \{0, 1\}$)

N is the number of samples

Murphy Decomposition:

$$\text{Brier Score} = \text{Reliability} - \text{Resolution} + \text{Uncertainty}$$

Definitions:

- Reliability (Calibration) : Discrepancy between predicted
- Resolution : How much the outcome frequency
- Uncertainty (Base Noise) : Irreducible variance of the target

Comparison of Brier Score, Log-Loss, and ECE on a reliability plot. Source: MetricGate, licensed under CC BY 4.0.

Three methods to fix a model's calibration:-

1. In Training Regularisation (During Training):-

- **Label Smoothing:** Modify the one-hot labels with a uniform distribution across all classes.
- **Focal Loss:** It replaces the cross-entropy with a modulated loss function.
- **Weight Decay/L2 Regularisation:** Works by penalising large parameter weights.

2. Post-Hoc Recalibration (After Training):-

- **Temperature Scaling:** As softmax is **shift-invariant**, we divide the raw logits by a scalar parameter before softmax ($\hat{p} = \text{Softmax}(z / T)$).

It basically flattens the distribution. It is the multi-class extension of **Platt Scaling**. If it is required to learn a separate scale per class, **Vector Scaling** is used.

- **Isotonic Regression**: It is a non-parametric method that basically learns a function that maps uncalibrated probabilities to calibrated ones. Works best with binary classifiers and requires a larger validation set. Not suitable for neural network-based models/LLMs.

3. RL and Objective-Based Calibration:-

- **ECE Penalty Integration**: It works by directly adding a penalty for ECE (which we already discussed above) into the policy gradient reward function during fine-tuning.

$$R_{\text{total}} = R_{\text{accuracy}} - \lambda \cdot \text{ECE}$$

Where:

R_{total} = Total combined reward signal

R_{accuracy} = Accuracy or task reward signal

λ (lambda) = Calibration penalty weight

ECE = Expected Calibration Error

- **RLCD**: Based on "Proper Scoring Rules as rewards". We are going to discuss this part in detail.

RLCD

Let's first theoretically understand what RLCD does.

The problem:-

In a general RLVR-based system, the model gets a point for a correct answer (+1) and nothing for an incorrect one (0). We can observe that there is no penalty for bluffing; the model always shows 100% certainty

for every question. This leads to a situation where the model outputs extreme scores on the answers where it is actually uncertain or guessing, because the reward function does not penalise false confidence (bluff).

Let's understand it with an example:-

Let's assume there are two models, A and B. Both are designed for multiclass classification with 4 classes (i, j, k, l). And we have a standard RLVR that gives +1 for a correct option and 0 for an incorrect option.

We can observe that the environment gives the same reward (+1) to 25% (model A), which might be a random guess or when it claims 99% certainty (model B).

Then, using policy gradient algorithms like PPO and GRPO, the optimiser updates the weights using the log-prob gradient. So whenever the model gives a correct answer ($R = +1$), the policy-gradient update (gradient ascent on expected reward) increases the logits of that choice (towards $+\infty$) over multiple training steps, which eventually leads to higher probabilities after softmax.

Get Utkarsh Ankit's stories in your inbox

Join Medium for free to get updates from this writer.

Remember me for faster sign in

This leads to the model saying 99% confident, yielding max rewards, and

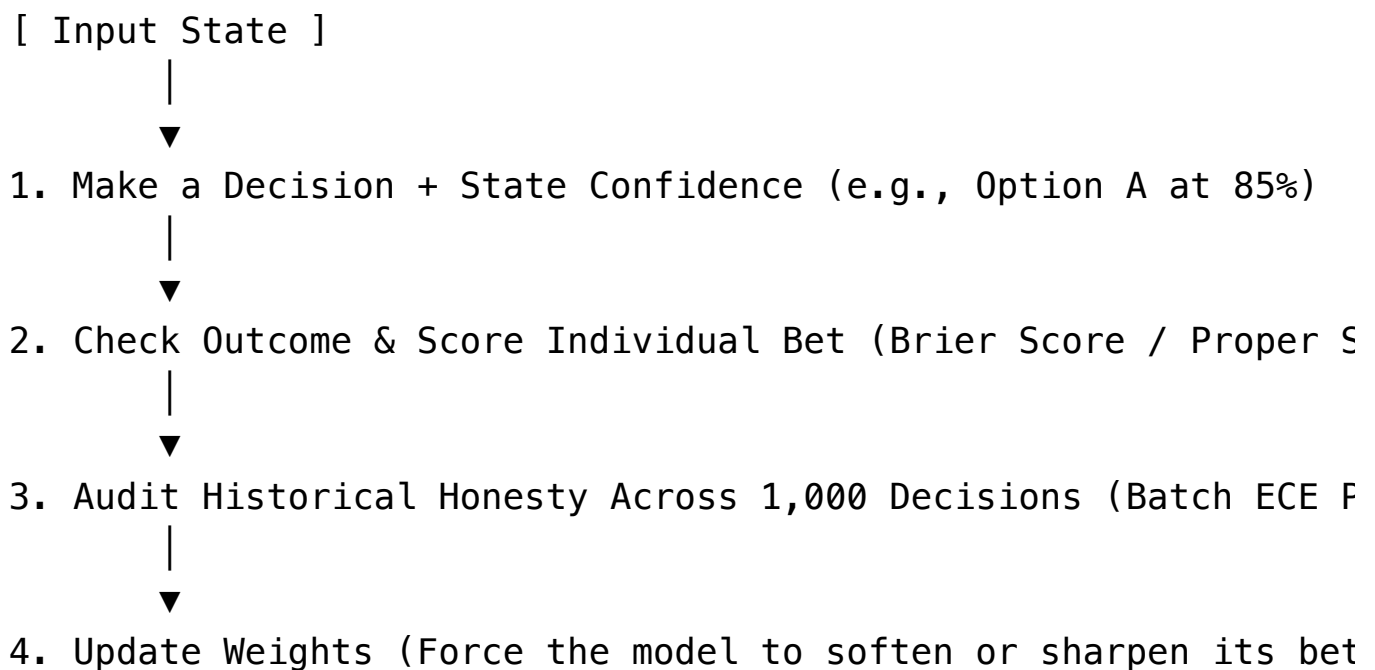
yet it never checks whether the probability matches the historical accuracy.

The solution:-

In RLCD, the model must give its answer and place a confidence score (0%-100%).

- **Correct Answer + High Bet (95%):** Big Reward (+10 points)
- **Correct Answer + Low Bet (60%):** Small Reward (+3 points)
- **Wrong Answer + Low Bet (60%):** Small Penalty (-2 points)
- **Wrong Answer + High Bet (95%): MASSIVE PENALTY (-100 points)**

Steps of RLCD pipeline:-



So, by repeating these steps multiple times during training, the model develops a calibrated score, so when it says it's 88.4% sure, it means that across thousands of real-world cases, it will be 88.4% correct.

In simple language, RLCD replaces the binary (+1/0) rewards with Proper Scoring Rules (like the Brier Score) and a batch calibration penalty. That gives us a properly aligned and calibrated model.

Architecture

1. The Actor/Policy Network (π_θ)

The Jev model acts as the Actor Network, that ingest the context and outputs the decision $\hat{y}$ along with the calibrated probability distribution $\hat{p}$.

Input State (s) $\longrightarrow$ Transformer Encoder $\longrightarrow$ Latent Vector $h(s)$

2. The Critic Network (V_ϕ)

The main role of a Critic Network is to provide advantage estimation (which can differ if PPO or GRPO is used during training). It is trained alongside the Actor network. The actor produces the decision probabilities; the critic predicts the expected calibrated return for state s .

- If PPO:-

$$A(s, y) = R_{\text{calibrated}}(P, y^*) - V_\phi(s)$$

Where:

- $A(s, y)$ = Calibrated Advantage Function
- $R_{\text{calibrated}}(\dots)$ = Proper Scoring Reward minus Batch ECE Per
- $V_\phi(s)$ = Critic's Expected Baseline Value for stat

- If GRPO:-

Here we remove the critic network altogether to save GPU memory during training. So, to calculate the advantage, the model samples a group of G

decisions for the same state and takes the average, which acts as the baseline.

$$A_i = (R_i - \text{mean}(R_1, \dots, R_G)) / \text{std}(R_1, \dots, R_G)$$

Where:

A_i = Group-relative advantage for sample i
 R_i = Calibrated reward for sample i
 $\text{mean}(R_1, \dots, R_G)$ = Mean reward across sample group G
 $\text{std}(R_1, \dots, R_G)$ = Standard deviation of rewards across G

3. The Reward Function

This is the interesting part: traditional RLHF trains a reward model (a neural network) that approximates human preferences (which is prone to reward hacking, where the generator usually tricks the reward model using a certain type of answer to get a better reward, even if the answer is bad).

In RLCD, there is **no learned reward model**. The reward engine is designed to be **deterministic** (a closed-form mathematical verifier), which evaluates the Actor's response against the ground truth using the Proper Scoring Rule. Because the objective truth already exists, we don't need a separate NN to guess the score. Then we evaluate the prediction directly against the ground truth.

$$R_{\text{Brier}} = 1 - \sum_{k=1}^K (p_k - y_{k*})^2$$

Where:

K = Total number of schema options
 p_k = Predicted probability for option k
 y_{k*} = Ground-truth binary target (1 if correct,
 $\sum (p_k - y_{k*})^2$ = Brier Loss (Mean Squared Error)
 $1 - \text{Brier Loss}$ = Final Calibrated Reward Signal

It gives better alignment; if the Actor is only 60% certain, claiming 90% will lower its expected mathematical payoff over multiple training steps

Also, there is no reward-hacking problem since the reward mechanism is a fixed equation that depends entirely on the objective ground truth, so it can't trick the evaluator.

FYI: TypeSafe hasn't publicly disclosed their exact reward mechanism; this is my reconstruction based on the standard calibration methods.

Let's take an example. Assume a binary classification with 2 outcomes, Option A (K=1) and Option B (K=2).

4. The Batch Calibration (Global Monitor)

It acts as a supervisor during training:

- Observes a mini-batch of N trajectories.
- Does the binning (predictions by reported confidence eg. 80%–90%)
- Computes the **Expected Calibration Error (ECE)** across the batch.

1. Average Bin Confidence:

$$\text{conf}(B_m) = (1 / |B_m|) * \sum_{(i \in B_m)} \hat{p}_i$$

2. Empirical Bin Accuracy:

$$\text{acc}(B_m) = (1 / |B_m|) * \sum_{(i \in B_m)} 1(\hat{y}_i = y_i^*)$$

3. Expected Calibration Error (ECE):

$$ECE(B) = \sum_{m=1}^M (|B_m| / N) * |acc(B_m) - conf(B_m)|$$

Interpretation:

- $ECE \approx 0.0$: Model confidence perfectly matches empirical accuracy
- $ECE > 0.0$: Presence of systemic miscalibration across the data

- After that, it turns the calibration gap into a per-sample reward adjustment. And just subtracting a single $-\lambda \cdot ECE$ from every sample would not work (λ is a scaling variable). RL learns from how each sample compares to the average; therefore, a penalty that is shared by everyone cancels out. So instead of that, each sample is adjusted based on the bin it falls in:

$$R_i \leftarrow R_i - \lambda \cdot (conf(B_m) - acc(B_m)) \quad \text{for sample } i \text{ in bin } m$$

- Overconfident bin ($conf > acc$): reward goes down $\rightarrow$ the model lowers its confidence on those predictions.
- Underconfident bin ($conf < acc$): reward goes up $\rightarrow$ the model raises its confidence.

After many updates, it pulls each confidence level toward the actual accuracy it achieves.

So, this is about the architecture.

TypeSafe's claim:-

1. PERFECT CALIBRATION (No Bluffing)
2. SUB-MILLISECOND (No token-by-token decoding loops)
3. SCHEMA GUARANTEE (Outputs bounded, typed tensors)
4. NO REWARD HACKING (Closed-form Proper Scoring Rules)

5. COMPUTE EFFICIENCY (Compact specialised models outperform 70B+ LLMs on structured enterprise decision tasks.)

Conclusion

I tried to sum up the concepts according to my understanding, since the full technical specifications from TypeSafe remain proprietary.

RLCD is a step towards the kind of models “that know what they don’t know”, which is exactly what is needed in automation.

Acknowledgements & Let’s Connect

I hope you enjoyed this breakdown. I tried to simplify things and connect the dots.

A special thanks to Corgi Cafe, SF, for providing a beautiful workplace and late-night fuel (Espresso shots) that made this writing possible.

Connect with me on [LinkedIn](#) and [X](#) here.

I would love to hear your thoughts, and I am always open to discussion.

References & Further Reading

Official TypeSafe AI Resources

- [Introducing System One Models and Jev](#) — *TypeSafe AI Blog* (Sept 2026)
- [The Bitterest Lesson](#) — *TypeSafe AI Blog*
- [TypeSafe AI Technical Documentation](#) — *Official API & Schema Specs*
- [TypeSafe Benchmarks & Evaluation Suite](#) — *Official Calibration & Performance Metrics*

Academic Papers

- [JEV-as-a-Judge: Accept When Confident, Escalate When Unsure](#) — *Li et al. (arXiv:2609.26550, Sept 2026)*
- [On Calibration of Modern Neural Networks](#) — *Guo et al. (ICML 2017)*
- [Verification of Forecasts Expressed in Terms of Probability](#) — *Brier (Monthly Weather Review, 1950)*

Industry Technical Overviews

- [What Is Jev? TypeSafe AI's System One Model Explained](#) — *You.com Technical Resources*
- [RLCD: Reinforcement Learning for Calibrated Decisions](#) — *Sanity AI Architecture Glossary*